

Porto-Star - Project Summary

Round-1 presentation reference | 8 October 2026

Presentation opening

Porto-Star transforms scattered student evidence into explainable academic and career intelligence. Students see their standing and improvement areas, faculty see who may need support and why, and recruiters explore relevant professional evidence through a separate view. The prototype combines deterministic mathematics with readable explanations instead of asking AI to invent scores.

The problem we chose

The hackathon's Smart Campus Analytics challenge asks for integrated academic performance, attendance, LMS learning, engagement, placement preparation, skills, and feedback evidence. It calls for a success score, academic/placement review indicators, and an interactive faculty dashboard. Explainable drivers and student segmentation are additional opportunities.

Each source answers a different question. Grades show assessed academic performance; attendance shows participation; assignments show learning progress; placement assessments show one kind of career practice. A student can perform well academically while still needing career preparation. Looking at only one measure hides such gaps. Looking across seven separate systems makes it difficult to form an overall picture.

A dashboard can also contain many numbers without helping anyone act. A student needs to know what caused a flag and what could improve. Faculty need to distinguish academic concerns from career-practice concerns and prioritize human review. Recruiters need professional evidence without unrelated private support information.

Why we chose this problem

We chose it because it connects data to a practical support decision. The three user groups have clear needs and different access boundaries. A useful prototype can be demonstrated with synthetic evidence while remaining honest about what its calculations do and do not mean.

The challenge also makes responsible design concrete: visible formulas, missing-data handling, consent, and faculty judgment matter as much as visual polish. A simple modular Round-1 stack lets the team inspect the logic and improve it in reviewable modules. A larger synthetic dataset lets us exercise search, filters, reporting, and cohort analysis before importing private campus data.

Problems the prototype addresses

1. Fragmented evidence: source-aware student and faculty views bring seven categories into a common interface. Live automated connectors remain future work.
2. Unexplained scores: score cards reveal indicator contributions, hover explanations, and a calculation dialog.
3. Unclear improvement priorities: a combined review index is accompanied by full reasons and specific areas to discuss or practice.

4. Limited cohort visibility: faculty can filter and inspect 500 synthetic students, subject evidence, academic/career review groups, and meaningful segments.
5. Hard-to-read reports: English interpretation, visual bars, method notes, and limitations make downloadable reports more useful than raw numerical dumps.
6. Privacy conflicts: recruiter discovery uses a separate professional projection. Live recruiter consent access remains unfinished rather than being implied by the preview.
7. Confusing onboarding: a new registered student starts empty. Public Aaman preview is a separate demonstration path.

Current results and scope

The 500-record synthetic demo has a mean success score of 72.6/100 under the current illustrative rules. A total of 193 students trigger at least one check. Academic review includes 147 and career review includes 89; these groups overlap and must not be added as distinct people. Aaman's score is 77/100, and his coding-practice check gives a 25% combined review index: one triggered check out of four assessed checks.

These describe synthetic records, not actual student outcomes. Equal weights and thresholds are prototype assumptions, not validated institutional policy. Student own-record persistence exists. Live faculty/recruiter queries, source imports, evidence verification, student doubts, and the richer effort simulator remain future modules.

Stack and why we used it

Vite, HTML, CSS, and vanilla JavaScript provide a small modular browser runtime with fast iteration. This follows the locked Round-1 architecture and avoids another framework/backend. Supabase PostgreSQL provides structured persistence and migrations; Supabase Auth manages email/password identities. Database roles and RLS enforce ownership instead of trusting a selected interface role.

Supabase Edge Functions and Vault keep the Gemini key and provider requests server-side. Gemini's purpose is brief English representation of computed evidence, with deterministic fallback when unavailable. jsPDF creates local downloadable vector reports and is loaded on demand. Vercel builds and hosts the frontend over HTTPS while Supabase remains the backend.

The Node test runner checks calculation and service behavior. GitHub provides history and collaboration. Codex assists implementation and validation, and the supplied Stitch designs guide interface hierarchy and theme. Human review is retained after each module. Offline Python prepares presentation documents; it is not an application server.

Evidence and positioning

The latest analytics checkpoint passed 36 automated tests, a production build, database access checks, provider/fallback checks, and visual review of generated reports. Actual signed-in narration and browser download capture were not confirmed in that checkpoint.

Our defensible claim is that Porto-Star demonstrates evidence, explanation, and a practical review action in the same product. It does not prove increased placement rates, predict individual failure, or claim completed integration with a college's systems.

Suggested presentation flow

Introduce the fragmentation problem. Open Aaman's preview and explain the score contributions and coding concern. Switch to Faculty to demonstrate cohort counts, filtering, segmentation, and student review. Switch to Recruiter to show professional discovery and its information boundary. Show a report, explain the architecture, and close with institutional imports and access validation as the next step.

References

Problem source: docs/reference/Hackathons p1.pdf, KPMG Smart Campus Analytics challenge.
Product/architecture: the four root specifications. Current evidence: docs/ANALYTICS_MODULE_02.md, source modules and tests. The brief specifies capabilities; it does not provide the prototype's chosen weights or validate its predictions.